

中山大学 计算机学院本科生实验报告

(2025/2026学年)

课程名称	并行程序设计 with 算法 (实验)	批改人	
实验	Lab 9 - CUDA Hello World 与 CUDA 矩阵转置	专业 (方向)	计算机科学与技术 (人工智能)
学号	23336103	姓名	雷颜玮
Email	leiyanyanwei2005@163.com	完成日期	2026年6月9日

1. 实验要求与目的

- 使用 CUDA 编写 Hello World 程序, 输入 `n, m, k` 三个整数, 创建 `n` 个线程块, 每个线程块维度为 `m x k`, 并由每个 GPU 线程输出自身二维线程编号和 block 编号。
- 观察 CUDA kernel 中多个线程 `printf` 的输出顺序, 分析其是否具有固定规律。
- 随机生成 `n x n` 单精度矩阵, 其中 `n` 范围为 `[512, 2048]`, 使用 CUDA 并行计算矩阵转置 `AT[i][j] = A[j][i]`。
- 实现并比较两种矩阵转置 kernel: 直接全局内存访问的 `naive` 版本, 以及使用 shared memory 分块的 `tiled` 版本。
- 扫描不同矩阵规模、线程块大小和访存方式, 采集运行时间、估算带宽和最大误差, 并分析这些因素对性能的影响。

2. 实验环境

- OS: Linux 服务器环境
- GPU: NVIDIA H20, 显存 97871 MiB
- GPU Driver: 580.105.08
- CUDA Toolkit: CUDA 12.6
- CUDA 编译器: `/usr/local/cuda/bin/nvcc`
- 编译参数: `-O3 -std=c++17 -Xcompiler -Wall, -Wextra`
- 实验运行 GPU: `CUDA_VISIBLE_DEVICES=6`

3. 实现说明

实现文件:

- `lab9/src/cuda_hello.cu`
- `lab9/src/matrix_transpose.cu`
- `lab9/run_experiments.py`

3.1 CUDA Hello World

`cuda_hello` 从命令行读取 `n, m, k`, 检查三个参数均在 `[1, 32]` 范围内。主机端先输出 `Hello World from the host!`, 随后启动 kernel:

```
hello_kernel<<<n, dim3(m, k)>>>
```

每个 GPU 线程使用 `threadIdx.x`、`threadIdx.y` 和 `blockIdx.x` 输出自身位置。kernel 启动后调用 `cudaGetLastError()` 和 `cudaDeviceSynchronize()` 检查并等待设备端执行结束。

3.2 naive 全局内存矩阵转置

`naive` 版本中，每个 CUDA 线程负责输出矩阵中的一个元素。线程根据二维 `block` 和二维 `thread` 计算原矩阵坐标 `(row, col)`，然后直接写入转置位置：

```
__global__ void transpose_naive(const float *input, float *output, int n)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < n && col < n) {
        output[col * n + row] = input[row * n + col];
    }
}
```

该实现简单直接，读入矩阵时相邻线程通常读取连续地址，但写出转置矩阵时会出现跨行、非连续写入，因此全局内存访问合并效果受限。

3.3 tiled shared-memory 分块矩阵转置

`tiled` 版本先将一个方形 `tile` 从全局内存读入 `shared memory`，经过 `__syncthreads()` 同步后，再按转置后的坐标写回全局内存：

```
__global__ void transpose_tiled(const float *input, float *output, int n)
{
    extern __shared__ float tile[];
    int tile_dim = blockDim.x;
    int block_rows = blockDim.y;

    int x = blockIdx.x * tile_dim + threadIdx.x;
    int y = blockIdx.y * tile_dim + threadIdx.y;

    for (int j = 0; j < tile_dim; j += block_rows) {
        if (x < n && y + j < n) {
            tile[(threadIdx.y + j) * tile_dim + threadIdx.x] =
                input[(y + j) * n + x];
        }
    }
    __syncthreads();

    x = blockIdx.y * tile_dim + threadIdx.x;
    y = blockIdx.x * tile_dim + threadIdx.y;

    for (int j = 0; j < tile_dim; j += block_rows) {
        if (x < n && y + j < n) {
            output[(y + j) * n + x] =
                tile[threadIdx.x * tile_dim + threadIdx.y + j];
        }
    }
}
```

该版本的目标是通过 `shared memory` 将全局内存中的非连续访问转换为块内访问，从而改善访存效率。不过它也引入了 `shared memory` 读写、同步和循环展开开销，因此最终性能需要结合实际 `block` 形状和矩阵规模观察。

4. 正确性验证

4.1 编译与静态测试

验证命令：

```
cd lab9
make -B all
make test
本次 `make -B all` 使用 `/usr/local/cuda/bin/nvcc` 重新编译 `cuda_hello` 和 `matrix_transpose`，编译通过。
`make test` 输出：

```text
.....

Ran 6 tests in 0.002s

OK
```

## 4.2 CUDA Hello World 运行验证

运行命令：

```
CUDA_VISIBLE_DEVICES=6 ./bin/cuda_hello 2 4 4
```

输出节选：

```
Hello World from the host!
Hello World from Thread (0, 0) in Block 0!
Hello World from Thread (1, 0) in Block 0!
Hello World from Thread (2, 0) in Block 0!
Hello World from Thread (3, 0) in Block 0!
...
Hello World from Thread (0, 0) in Block 1!
Hello World from Thread (1, 0) in Block 1!
...
Hello World from Thread (3, 3) in Block 1!
```

另外测试了 `1 2 2` 和 `4 8 8` 两组参数，均能输出 host 行和对应数量的 device 线程行。

## 4.3 单次矩阵转置正确性验证

运行命令：

```
CUDA_VISIBLE_DEVICES=6 ./bin/matrix_transpose --n 1024 --kernel tiled --block-x 32 --block-y 8 --repeats 20
CUDA_VISIBLE_DEVICES=6 ./bin/matrix_transpose --n 1024 --kernel naive --block-x 16 --block-y 16 --repeats 20
```

本次运行输出：

```
matrix_size=1024 kernel=tiled block_x=32 block_y=8 repeats=20 time_ms=0.021778
bandwidth_gb_s=385.194327 max_abs_error=0.000000
matrix_size=1024 kernel=naive block_x=16 block_y=16 repeats=20 time_ms=0.021414
bandwidth_gb_s=391.727417 max_abs_error=0.000000
```

两个 kernel 的 `max_abs_error` 均为 `0.000000`，说明 GPU 转置结果与 CPU 端校验结果一致。

## 5. 实验数据采集方式

---

统一脚本: `lab9/run_experiments.py`

实验设置:

- 矩阵规模: `512, 1024, 2048`
- kernel: `naive, tiled`
- block 配置: `16x16, 32x8, 32x16`
- 每组 kernel 重复次数: `20`
- 随机种子: `20260609`

输出文件:

- 汇总结果: `lab9/results/transpose_summary.csv`

带宽估算方式为每次转置读一次矩阵、写一次矩阵:

```
bandwidth_gb_s = 2 * n * n * sizeof(float) / time
```

其中 `time` 为单次 kernel 的平均运行时间。

## 6. 实验结果

---

### 6.1 批量矩阵转置性能数据

n	kernel	block	repeats	time_ms	bandwidth_gb_s	max_abs_error
512	naive	16x16	20	0.014725	142.423119	0.0
512	naive	32x8	20	0.016506	127.056992	0.0
512	naive	32x16	20	0.017974	116.674376	0.0
512	tiled	16x16	20	0.020194	103.852304	0.0
512	tiled	32x8	20	0.015747	133.176178	0.0
512	tiled	32x16	20	0.016512	127.007745	0.0
1024	naive	16x16	20	0.019611	427.745771	0.0
1024	naive	32x8	20	0.028778	291.497838	0.0
1024	naive	32x16	20	0.028571	293.603633	0.0
1024	tiled	16x16	20	0.021475	390.618395	0.0
1024	tiled	32x8	20	0.022416	374.224137	0.0
1024	tiled	32x16	20	0.024925	336.556685	0.0
2048	naive	16x16	20	0.040904	820.321525	0.0
2048	naive	32x8	20	0.073272	457.943414	0.0
2048	naive	32x16	20	0.081126	413.606819	0.0
2048	tiled	16x16	20	0.045973	729.875725	0.0
2048	tiled	32x8	20	0.054938	610.773520	0.0
2048	tiled	32x16	20	0.053645	625.492738	0.0

全部 18 组实验的 `max_abs_error` 均为 `0.0`，说明不同矩阵规模、kernel 和 block 配置下结果均正确。

## 6.2 最优配置汇总

n	最快 kernel	block	time_ms	bandwidth_gb_s
512	naive	16x16	0.014725	142.423119
1024	naive	16x16	0.019611	427.745771
2048	naive	16x16	0.040904	820.321525

按所有 18 组数据计算，三种 block 配置的平均时间为：

block	平均 time_ms
16x16	0.027147
32x8	0.035276
32x16	0.037126

按 kernel 汇总的平均时间为：

kernel	平均 time_ms
naive	0.035719
tiled	0.030647

## 7. 结果分析

1. 从平均时间看，`tiled` 版本整体略快于 `naive` 版本，平均时间从 `0.035719 ms` 降至 `0.030647 ms`。这说明 shared memory 分块在部分 block 配置下改善了转置中的访存效率。
2. 但是最快单项并不是 `tiled`，而是 `naive + 16x16`。在 `n=512, 1024, 2048` 三个规模下，该配置均为最快。这说明 shared memory 优化并非无条件更快；当 block 形状、同步开销、shared memory 访问方式与硬件特性不匹配时，额外开销可能抵消访存收益。
3. 对于 `32x8` 和 `32x16`，`tiled` 通常明显快于对应的 `naive`。例如 `n=2048, 32x8` 下，`naive` 为 `0.073272 ms`，`tiled` 为 `0.054938 ms`；`n=2048, 32x16` 下，`naive` 为 `0.081126 ms`，`tiled` 为 `0.053645 ms`。这体现了 tiled 方法对不利全局写访问的缓解作用。
4. 矩阵规模从 `512` 增加到 `2048` 时，数据量按 `n^2` 增长，运行时间整体上升。以最快配置 `naive 16x16` 为例，时间从 `0.014725 ms` 增至 `0.040904 ms`。增长幅度小于矩阵元素数量增长幅度，原因是较大矩阵提供了更多并行任务，GPU 的并行度和带宽利用率更充分。
5. 本实验中 `16x16` 是平均最快的 block 配置。它包含 256 个线程，二维形状与矩阵二维划分匹配较好；相比之下，`32x16` 含 512 个线程，单 block 资源占用更高，可能降低并发 block 数量，从而影响吞吐。
6. CUDA Hello World 的线程输出顺序通常没有稳定规律。host 端输出发生在 kernel 启动之前，因此通常先出现；device 端各 block 和 warp 的调度由 GPU 硬件和运行时决定，`printf` 还经过设备端缓冲，不同线程的缓冲刷新顺序不构成程序语义保证。因此即使本次输出看起来较整齐，也不能依赖该顺序。

## 8. 结论

1. Lab9 的 CUDA Hello World 程序已完成，能够根据输入 `n, m, k` 创建指定数量和形状的线程块，并输出 host 与 device 端信息。
2. Lab9 的 CUDA 矩阵转置程序已完成，支持 `naive` 和 `tiled` 两种 kernel，全部批量实验结果的 `max_abs_error` 均为 `0.0`。
3. 性能实验表明，shared memory 分块版本在部分 block 配置下能明显改善性能，但最佳配置仍取决于线程块形状、同步开销、访存模式和 GPU 硬件特性。
4. 本次实验中综合表现最好的线程块配置为 `16x16`，三个矩阵规模下最快结果均为 `naive + 16x16`。
5. 矩阵规模增大后运行时间增加，但 GPU 并行度和带宽利用率也提高，因此时间增长低于元素数量的平方级增长。